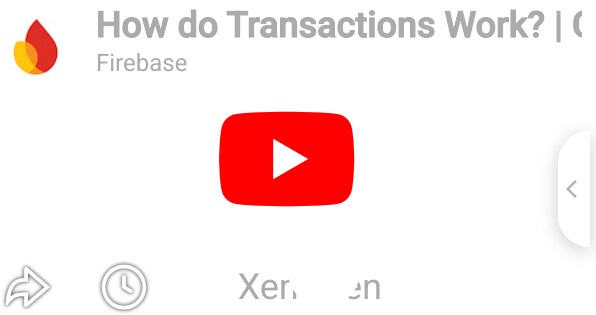
Transactions and batched writes

Cloud Firestore supports atomic operations for reading and writing data. In a set of atomic operations, either all of the operations succeed, or none of them are applied. There are two types of atomic operations in Cloud Firestore:



- **Transactions**: a transaction (#transactions) is a set of read and write operations on one or more documents.
- **Batched Writes**: a batched write (#batched-writes) is a set of write operations on one or more documents.

Updating data with transactions

Using the Cloud Firestore client libraries, you can group multiple operations into a single transaction. Transactions are useful when you want to update a field's value based on its current value, or the value of some other field.

A transaction consists of any number of `get()` operations followed by any number of write operations such as `set()`, `update()`, or `delete()`. In the case of a concurrent edit, Cloud Firestore runs the entire transaction again. For example, if a transaction reads documents and another client modifies any of those documents, Cloud Firestore retries the transaction. This feature ensures that the transaction runs on up-to-date and consistent data.

Transactions never partially apply writes. All writes execute at the end of a successful transaction.

When using transactions, note that:

- Read operations must be executed before write operations.
- A function calling a transaction (transaction function) might run more than once if a concurrent edit affects a document that the transaction reads.
- Transaction functions should not directly modify application state.
- Transactions will fail when the client is offline.

The following example shows how to create and run a transaction:

Web modular API | Web namespaced API | Swift (#swift) Objective-C (#objective-c) | Kotlin Android | Java Android

```
import { runTransaction } from "firebase/firestore";

try {
  await runTransaction(db, async (transaction) => {
    const sfDoc = await transaction.get(sfDocRef);
    if (!sfDoc.exists()) {
      throw "Document does not exist!";
    }

    const newPopulation = sfDoc.data().population + 1;
    transaction.update(sfDocRef, { population: newPopulation });
  });
  console.log("Transaction successfully committed!");
} catch (e) {
  console.log("Transaction failed: ", e);
}
```

Passing information out of transactions

Do not modify application state inside of your transaction functions. Doing so will introduce concurrency issues, because transaction functions can run multiple times and are not guaranteed to run on the UI thread. Instead, pass information you need out of your transaction functions. The following example builds on the previous example to show how to pass information out of a transaction:

Web | Web | Kotlin | Java

```
import { doc, runTransaction } from "firebase/firestore";

// Create a reference to the SF doc.
const sfDocRef = doc(db, "cities", "SF");

try {
  const newPopulation = await runTransaction(db, async (transaction) => {
    const sfDoc = await transaction.get(sfDocRef);
    if (!sfDoc.exists()) {
      throw "Document does not exist!";
    }

    const newPop = sfDoc.data().population + 1;
    if (newPop <= 1000000) {
      transaction.update(sfDocRef, { population: newPop });
      return newPop;
    } else {
      return Promise.reject("Sorry! Population is too big");
    }
  });

  console.log("Population increased to ", newPopulation);
} catch (e) {
  // This will be a "population is too big" error.
  console.error(e);
}
```

!42352f058406f1aeb8adcbb/snippets/firestore-next/test-firestore/transaction_promise.js#L8-L33)

Transaction failure

A transaction can fail for the following reasons:

- The transaction contains read operations after write operations. Read operations must always be executed before any write operations.
- The transaction read a document that was modified outside of the transaction. In this case, the transaction automatically runs again. The transaction is retried a finite number of times.
- The transaction exceeded the maximum request size of 10 MiB.

Transaction size depends on the sizes of documents and index entries modified by the transaction. For a delete operation, this includes the size of the target document and the sizes of the index entries deleted in response to the operation.

- The transaction exceeded the lock deadline (20 seconds). Cloud Firestore automatically releases locks if a transaction cannot complete in time.
- The transaction exceeds the 270-second time limit (https://firebase.google.com/docs/firestore/quotas#writes_and_transactions) or the 60-second idle expiration time. If no activity (reads or writes) occurs within the transaction, it will time out and fail.

A failed transaction returns an error and does not write anything to the database. You do not need to roll back the transaction; Cloud Firestore does this automatically.

Batched writes

If you do not need to read any documents in your operation set, you can execute multiple write operations as a single batch that contains any combination of **`set()`**, **`update()`**, or **`delete()`** operations. Each operation in the batch counts separately towards your Cloud Firestore usage. A batch of writes completes atomically and can write to multiple documents. The following example shows how to build and commit a write batch:

Web modular API | Web namespaced API | Swift (#swift) Objective-C (#objective-c) | Kotlin Android | Java Android

```
import { writeBatch, doc } from "firebase/firestore";

// Get a new write batch
const batch = writeBatch(db);

// Set the value of 'NYC'
const nycRef = doc(db, "cities", "NYC");
batch.set(nycRef, {name: "New York City"});

// Update the population of 'SF'
const sfRef = doc(db, "cities", "SF");
batch.update(sfRef, {"population": 1000000});

// Delete the city 'LA'
```

```
const laRef = doc(db, "cities", "LA");
batch.delete(laRef);

// Commit the batch
await batch.commit();
```

f4d90af442352f058406f1aeb8adcbb/snippets/firestore-next/test-firestore/write_batch.js#L8-L26)

Like transactions, batched writes are atomic. Unlike transactions, batched writes do not need to ensure that read documents remain un-modified which leads to fewer failure cases. They are not subject to retries or to failures from too many retries. Batched writes execute even when the user's device is offline.

A batched write with hundreds of documents might require many index updates and might exceed the limit on transaction size. In this case, reduce the number of documents per batch. To write a large number of documents, consider using a bulk writer or parallelized individual writes instead.

Note: For bulk data entry, use a server client library (/docs/firestore/client/libraries#server_client_libraries) with parallelized individual writes. Batched writes perform better than serialized writes but not better than parallel writes. You should use a server client library for bulk data operations and not a mobile/web SDK.

Data validation for atomic operations

For mobile/web client libraries, you can validate data using Cloud Firestore Security Rules (https://firebase.google.com/docs/firestore/security/get-started). You can ensure that related documents are always updated atomically and always as part of a transaction or batched write. Use the **`getAfter()`** (https://firebase.google.com/docs/reference/rules/rules.firestore#.getAfter) security rule function to access and validate the state of a document after a set of operations completes but *before* Cloud Firestore commits the operations.

For example, imagine that the database for the **`cities`** example also contains a **`countries`** collection. Each **`country`** document uses a **`last_updated`** field to keep track of the last time any city related to that country was updated. The following security rules require that an update to a **`city`** document must also atomically update the related country's **`last_updated`** field:

```
service cloud.firestore {
  match /databases/{database}/documents {
    // If you update a city doc, you must also
    // update the related country's last_updated field.
    match /cities/{city} {
      allow write: if request.auth != null &&
        getAfter(
          /databases/$(database)/documents/countries/$(request.resource.data.cou
        ).data.last_updated == request.time;
    }

    match /countries/{country} {
      allow write: if request.auth != null;
    }
  }
}
```

Security rules limits

In security rules for transactions or batched writes, there is a limit (https://firebase.google.com/docs/firestore/quotas#security_rules) of 20 document access calls for the entire atomic operation in addition to the normal 10 call limit for each single document operation in the batch.

For example, consider the following rules for a chat application:

```
service cloud.firestore {
  match /databases/{db}/documents {
    function prefix() {
      return /databases/{db}/documents;
    }
    match /chatroom/{roomId} {
      allow read, write: if request.auth != null && roomId in get(/$(prefix())/u
                         || exists(/$(prefix())/admins/$(request.auth.uid));
    }
    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId
                         || exists(/$(prefix())/admins/$(request.auth.uid));
    }
```

```
    match /admins/{userId} {
      allow read, write: if request.auth != null && exists(/$(prefix())/admins/$
    }
  }
}
```

The snippets below illustrate the number of document access calls used for a few data access patterns:

```
// 0 document access calls used, because the rules evaluation short-circuits
// before the exists() call is invoked.
db.collection('user').doc('myuid').get(...);

// 1 document access call used. The maximum total allowed for this call
// is 10, because it is a single document request.
db.collection('chatroom').doc('mygroup').get(...);

// Initializing a write batch...
var batch = db.batch();

// 2 document access calls used, 10 allowed.
var group1Ref = db.collection("chatroom").doc("group1");
batch.set(group1Ref, {msg: "Hello, from Admin!"});

// 1 document access call used, 10 allowed.
var newUserRef = db.collection("users").doc("newuser");
batch.update(newUserRef, {"lastSignedIn": new Date()});

// 1 document access call used, 10 allowed.
var removedAdminRef = db.collection("admin").doc("otheruser");
batch.delete(removedAdminRef);

// The batch used a total of 2 + 1 + 1 = 4 document access calls, out of a total
// 20 allowed.
batch.commit();
```

For more information on how to resolve latency issues caused by large writes and batched writes, errors due to contention from overlapping transactions, and other issues consider checking out the troubleshooting page (https://cloud.google.com/firestore/docs/troubleshooting).

Last updated 2026-05-19 UTC.